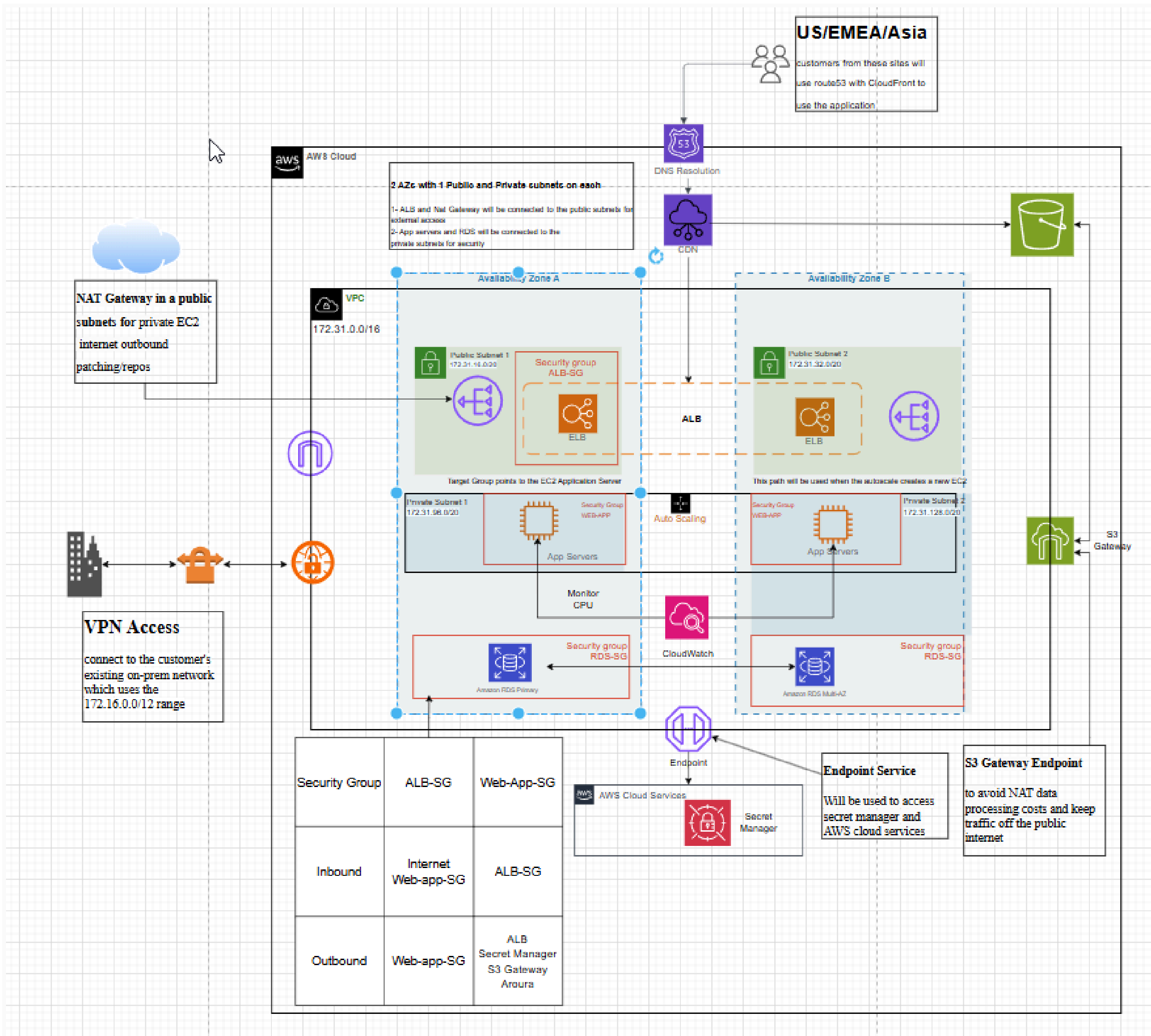
Delivering the required points

1- Diagram/specs should include additional well-architected security considerations (e.g., security groups/NACLs, NAT, VPC endpoints, clear routing boundaries, Secrets Manager placement)



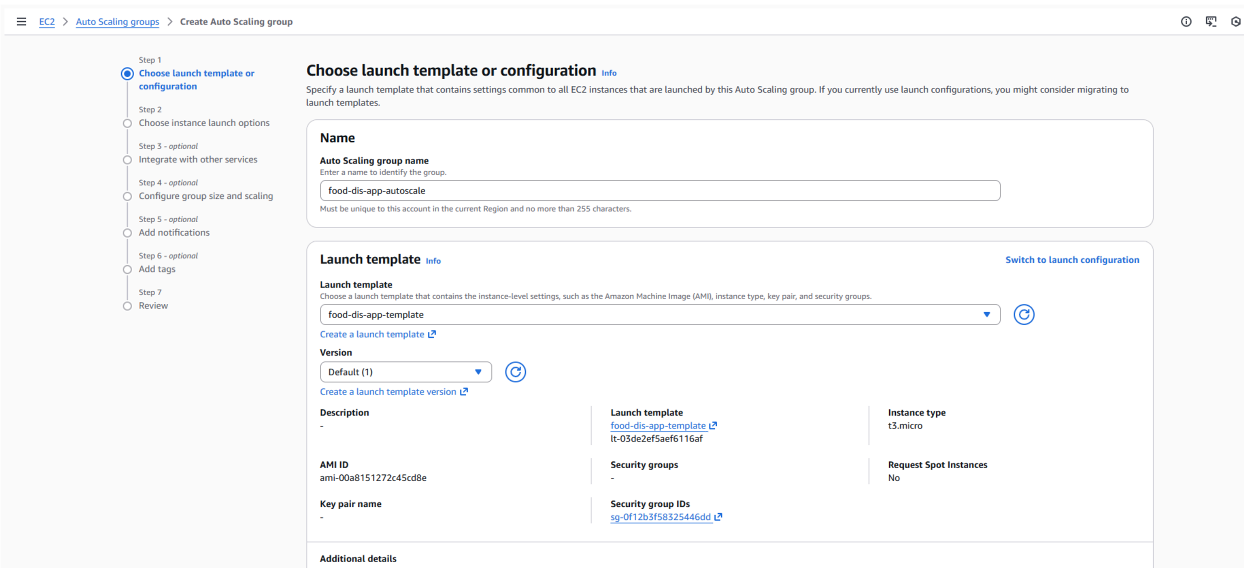
I have added the services below to the diagram:

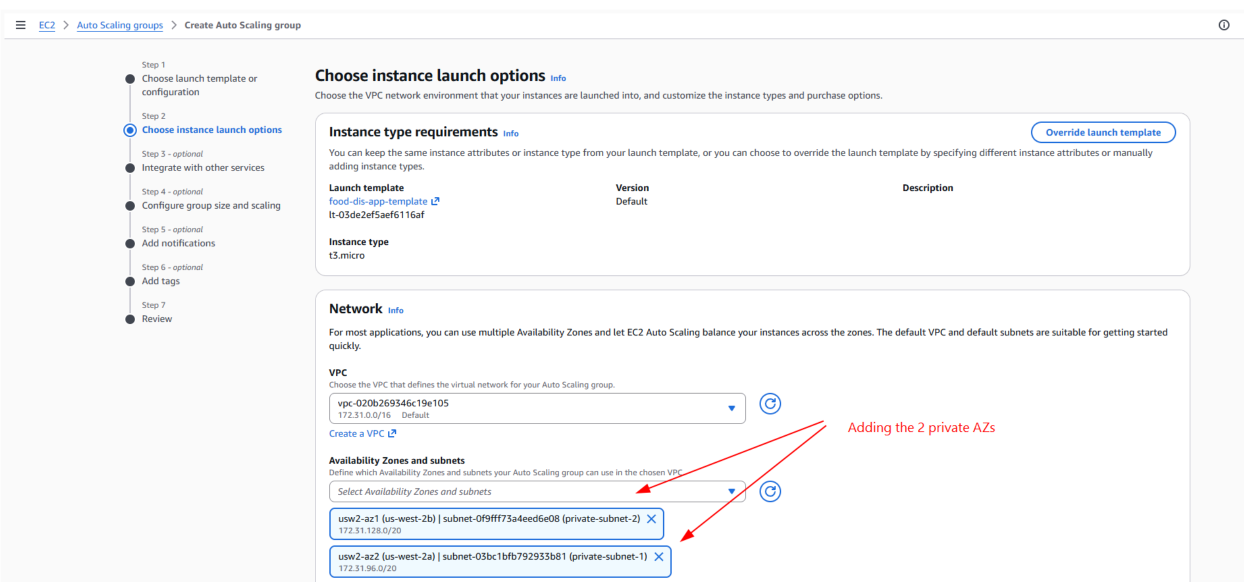
- security groups
- VPC Endpoint
- S3 Gateway
- Routing rules
- NAT Gateway

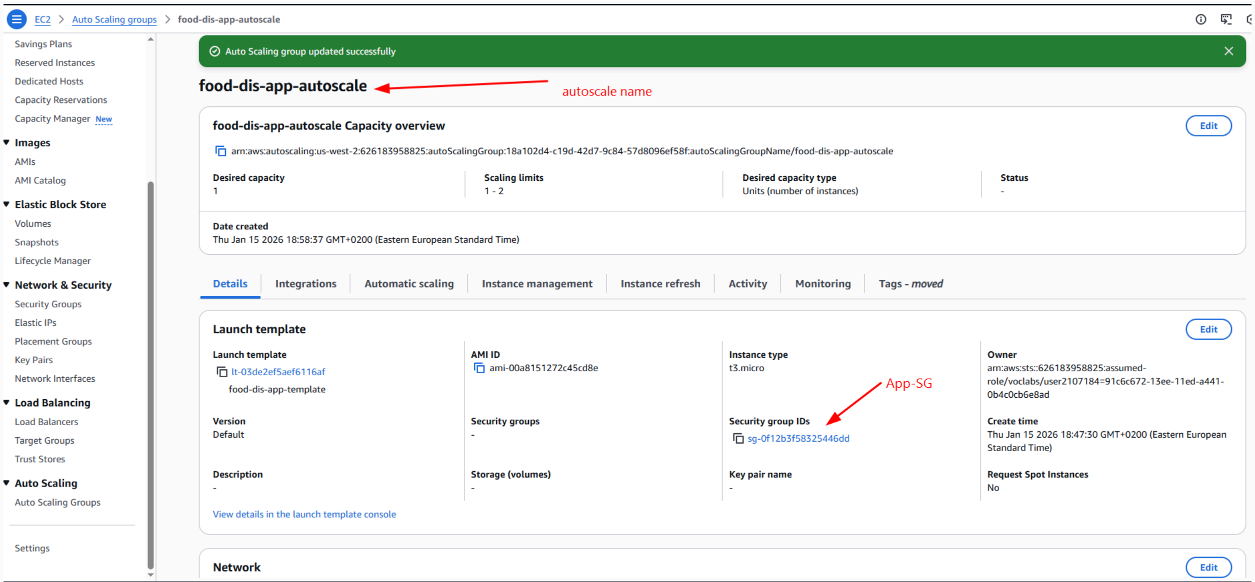
- Secret Manager

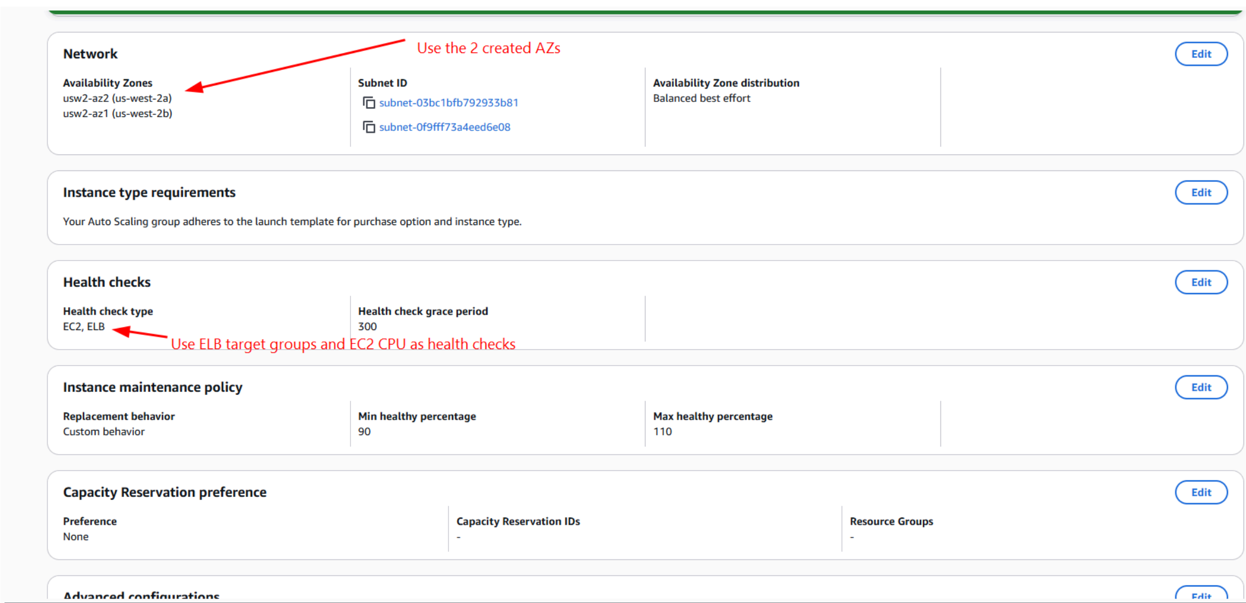
2- Architecture should reflect decoupling/scaling mechanisms in a well-architected way (e.g., Auto Scaling + CloudWatch, and/or asynchronous options like SQS/SNS/Lambda/EventBridge where appropriate)

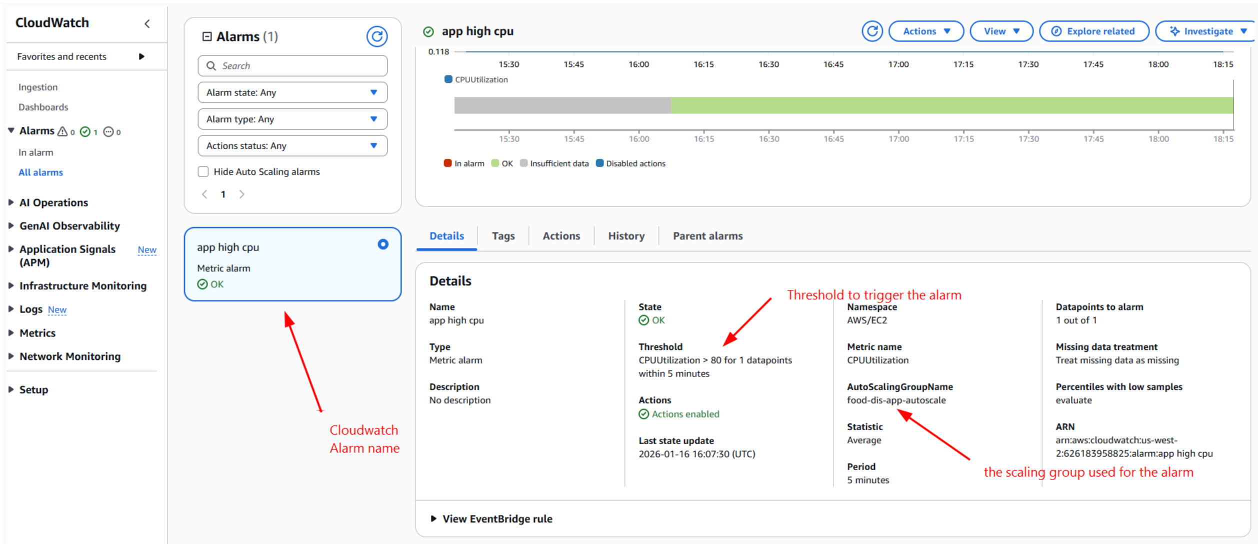
- added Cloudwatch service and configured alarms to to send SNS topic to an email for IT team
- Configured the autoscale policy to scale out based at 80% of the CPU load

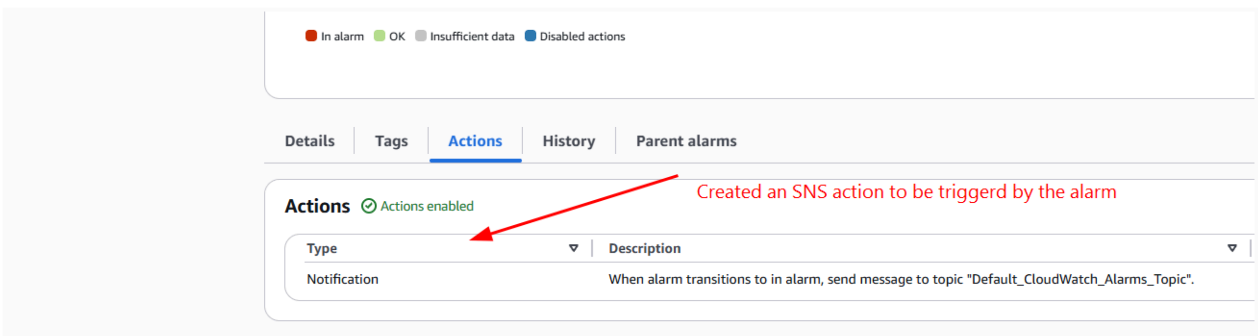





EC2 > Auto Scaling groups > food-dis-app-autoscale
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
Load Balancing
Load Balancers
Target Groups
Trust Stores
Auto Scaling
Auto Scaling Groups
Settings
Auto Scaling group updated successfully
food-dis-app-autoscale
autoscale name
food-dis-app-autoscale Capacity overview
Edit
arn:aws:autoscaling:us-west-2:626183958825:autoScalingGroup:18a102d4-c19d-42d7-9c84-57d8096ef58f:autoScalingGroupName/food-dis-app-autoscale
Desired capacity
1
Scaling limits
1 - 2
Desired capacity type
Units (number of instances)
Status
-
Date created
Thu Jan 15 2026 18:58:37 GMT+0200 (Eastern European Standard Time)
Details
Integrations
Automatic scaling
Instance management
Instance refresh
Activity
Monitoring
Tags - moved
Launch template
Edit
Launch template
lt-03de2ef5aef6116af
food-dis-app-template
AMI ID
ami-00a8151272c45cd8e
Instance type
t3.micro
Owner
arn:aws:sts::626183958825:assumed-role/voclabs/user2107184=91c6c672-13ee-11ed-a441-0b4c0cb6e8ad
App-SG
Version
Default
Security groups
-
Security group IDs
sg-0f12b3f58325446dd
Create time
Thu Jan 15 2026 18:47:30 GMT+0200 (Eastern European Standard Time)
Description
-
Storage (volumes)
-
Key pair name
-
Request Spot Instances
No
View details in the launch template console
Network
Edit


Network
Use the 2 created AZs
Edit
Availability Zones
usw2-az2 (us-west-2a)
usw2-az1 (us-west-2b)
Subnet ID
subnet-03bc1bfb792933b81
subnet-0f9fff73a4eed6e08
Availability Zone distribution
Balanced best effort
Instance type requirements
Edit
Your Auto Scaling group adheres to the launch template for purchase option and instance type.
Health checks
Edit
Health check type
EC2, ELB
Use ELB target groups and EC2 CPU as health checks
Health check grace period
300
Instance maintenance policy
Edit
Replacement behavior
Custom behavior
Min healthy percentage
90
Max healthy percentage
110
Capacity Reservation preference
Edit
Preference
None
Capacity Reservation IDs
-
Resource Groups
-
Advanced configurations
Edit





3- Pricing/right-sizing recommendations should be consistent, clearly justified, and reflect additional cost-avoidance levers (e.g., endpoints to reduce egress/NAT costs)

I already provided the sizing recommendation on my last submission but for the On-demand option, the recommendation below shows for the Saving Plan option

Assumptions

- Region: `us-east-1`
- Instance type: `r7g.large`
- OS: Amazon Linux (Linux pricing)
- Tenancy: Shared
- Pricing model: Saving Plan
- Hours/month: 730
- Instances: 2 (Multi-AZ)
- NAT Gateway: 1 per AZ (730 hrs)
- Data egress: 100 GB/month

I have attached a detailed screenshot showing how i used cost estimator

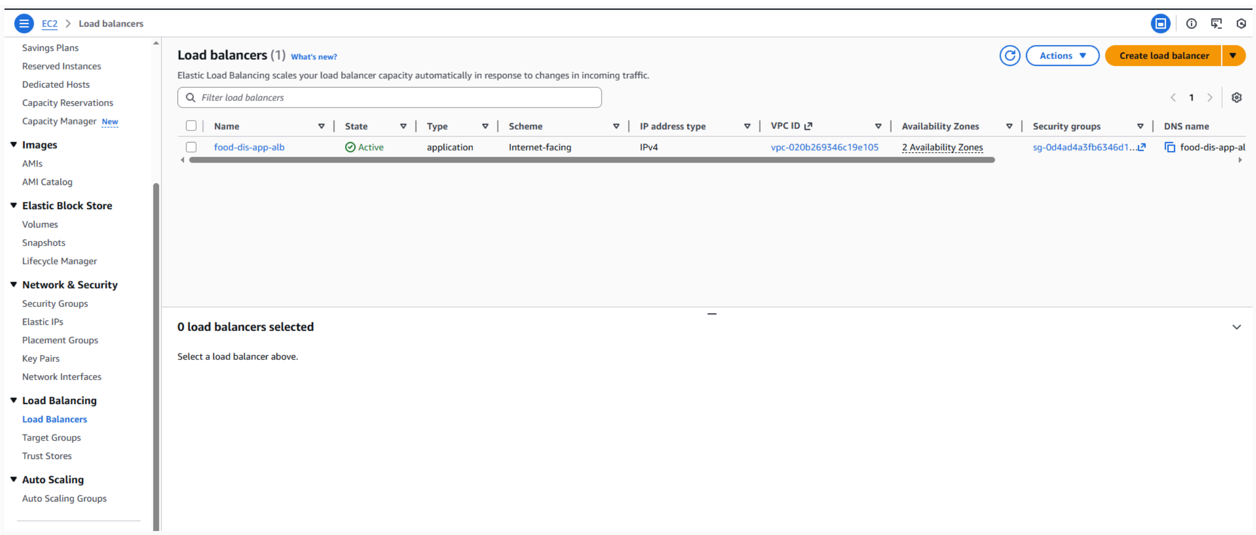
- Monthly cost: 47.58 USD
- Total 12 months cost: 570.96 USD

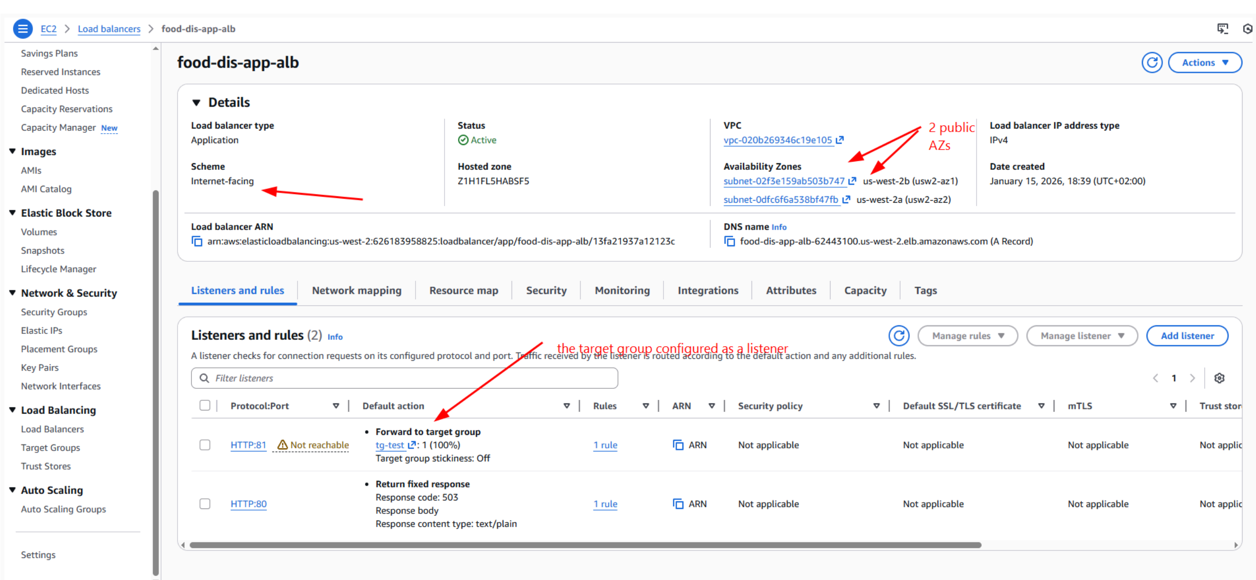
Justification:

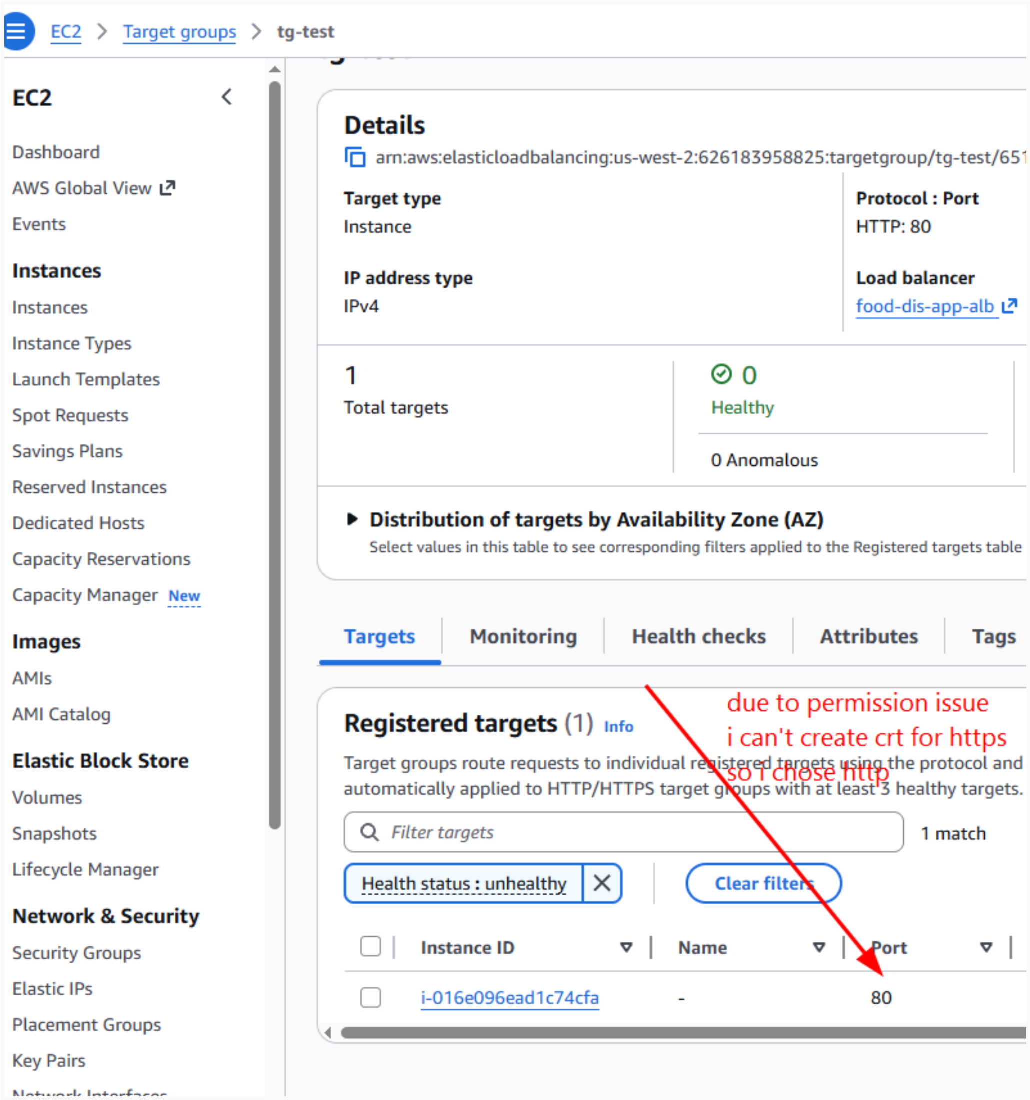
- We will use R7g Instances:
- Processor: AWS Graviton3 (ARM-based)
- Memory: DDR5 with 50% higher bandwidth than previous generations
- High memory-to-vCPU ratio (8:1)
- Excellent for PHP's memory-intensive operations
- Great for nginx caching and buffering
- Cost-effective ARM performance

4- Scaling strategy should include the Availability Triad (ALB + Auto Scaling + CloudWatch) or clearly explain an alternative

- Added Application load balancer which will be in 2 public AZs
- The ALB has target group points to the EC2 instance available on private AZ, the EC2 instance created by the auto scaling group
- The auto scaling group run 1 instance (for cost management, but we can make it 2 VMs)
- The auto scaling group check the ALB healthy targets and CPU metric
- Created a cloudwatch alarm based on the auto scaling group that will trigger an SNS action to the target mail






EC2 > Target groups > tg-test
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Details
arn:aws:elasticloadbalancing:us-west-2:626183958825:targetgroup/tg-test/651
Target type
Instance
Protocol : Port
HTTP: 80
IP address type
IPv4
Load balancer
food-dis-app-alb
1
Total targets
0
Healthy
0 Anomalous
Distribution of targets by Availability Zone (AZ)
Select values in this table to see corresponding filters applied to the Registered targets table
Targets
Monitoring
Health checks
Attributes
Tags
Registered targets (1) Info
Target groups route requests to individual registered targets using the protocol and
automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.
Filter targets
1 match
Health status : unhealthy
Clear filters
Instance ID
Name
Port
i-016e096ead1c74cfa
-
80
due to permission issue
i can't create crt for https
so i chose http